Department of Mathematics
and Computer Science

Advanced Machine Learning

Loss Functions

Lukas Galke Poech

Spring 2026

SDU

Keywords

- Generative Models
- Discriminative Models & Logistic Regression
- Information Theory
- Well Behaved Gradients

One Example

Decanter

Police uncover Italian wine fraud

Maggie Rosen
August 23, 2007

3 shares

Police have broken up an international counterfeit wine racket involving top Italian wines.

German and Italian police forces have uncovered a cross-border scam involving table wine from Puglia and Piedmont sold as Barolo, Brunello di Montalcino, Amarone and Chianti.

Unlabelled wine was brought into Germany, where it was given fake DOC and DOCG seals and phoney labels from well-known producers as well as non-existent wineries.

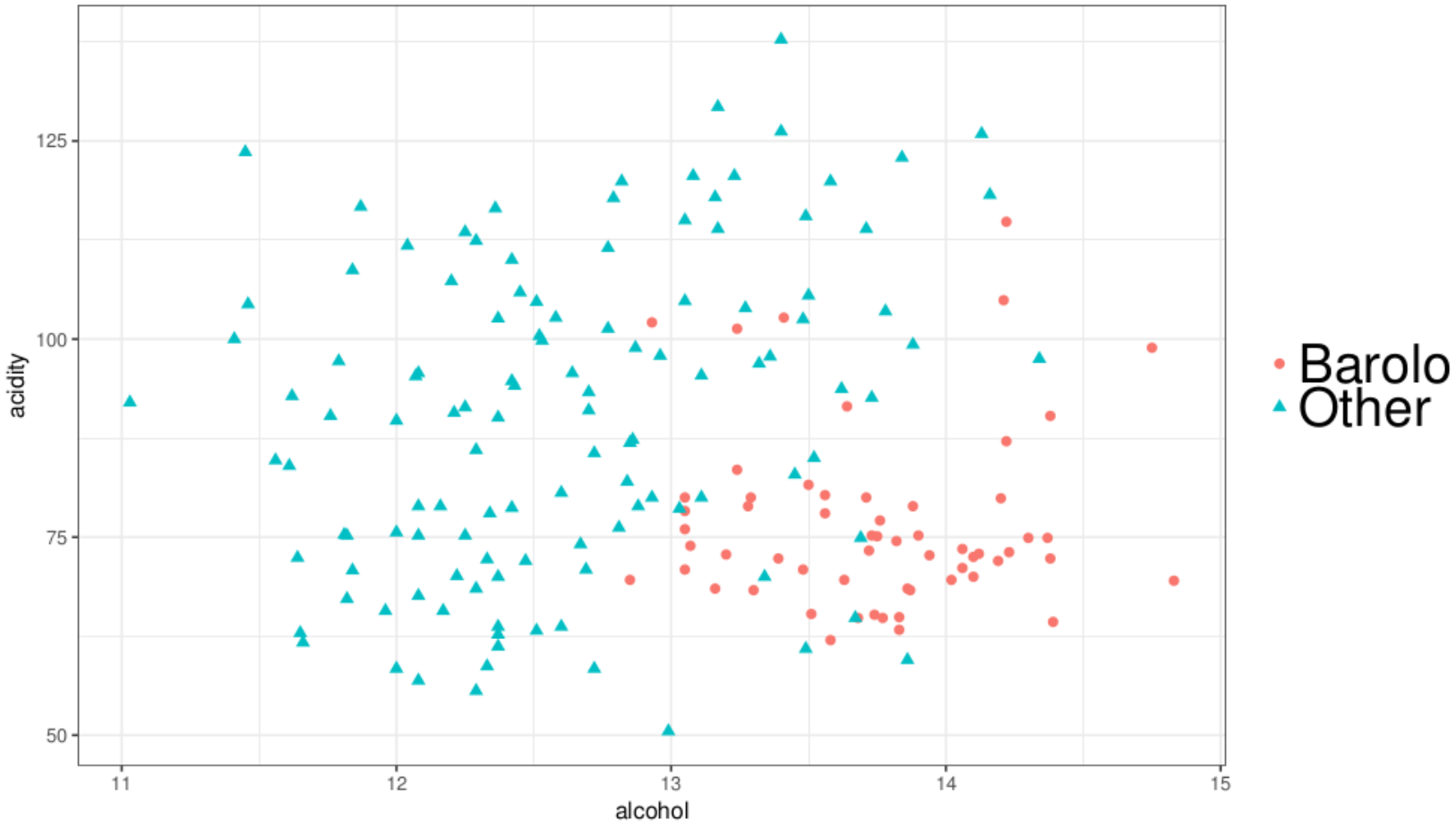
Ten people have been charged. It is thought the con could be worth €750,000.

➢ The following slides are adapted from Pierre-Alexandre Mattei and his "Machine Learning" course

One Example

- Would it be possible to create an AI device that learns to estimates the probability that a bottle of wine is fake or not?
- The device would need some information about the wine, for example some chemical properties:
  - alcohol content
  - acidity
- One of the wines the bad guys counterfeited was from the Barolo region.
  - Those wines have "pronounced tannins and acidity", and "moderate to high alcohol levels (Minimum 13%)". (Wikipedia)
  - So, can we use a computer to do so?

Our Dataset



Probabilistic Model for Classification

- There are two schools of modelling when it comes to building a probabilistic model of our data $\{x_n, t_n\}$.
- Both assume that the data are sampled from a distribution $p(x, t)$, or equivalently $p(x,\ C_k)$.
- The **generative school** builds models for the joint distribution $p(x, t)$.
  - This is quite hard, because this means we'll have in particular to learn the distribution of the data $p(x)$, which can be very complicated.
- The **discriminative school** builds models for the conditional distribution $p(t \mid x)$.
  - This is much easier, because we don't care about $p(x)$, but this is also often less powerful (in particular, all the training set needs to be labelled).

A Generative Model

➢ So, we want to define a distribution $p(x, C_k)$ Using the product rule, we can write

$$p(x, C_k) = p(x \mid C_k) p(C_k)$$

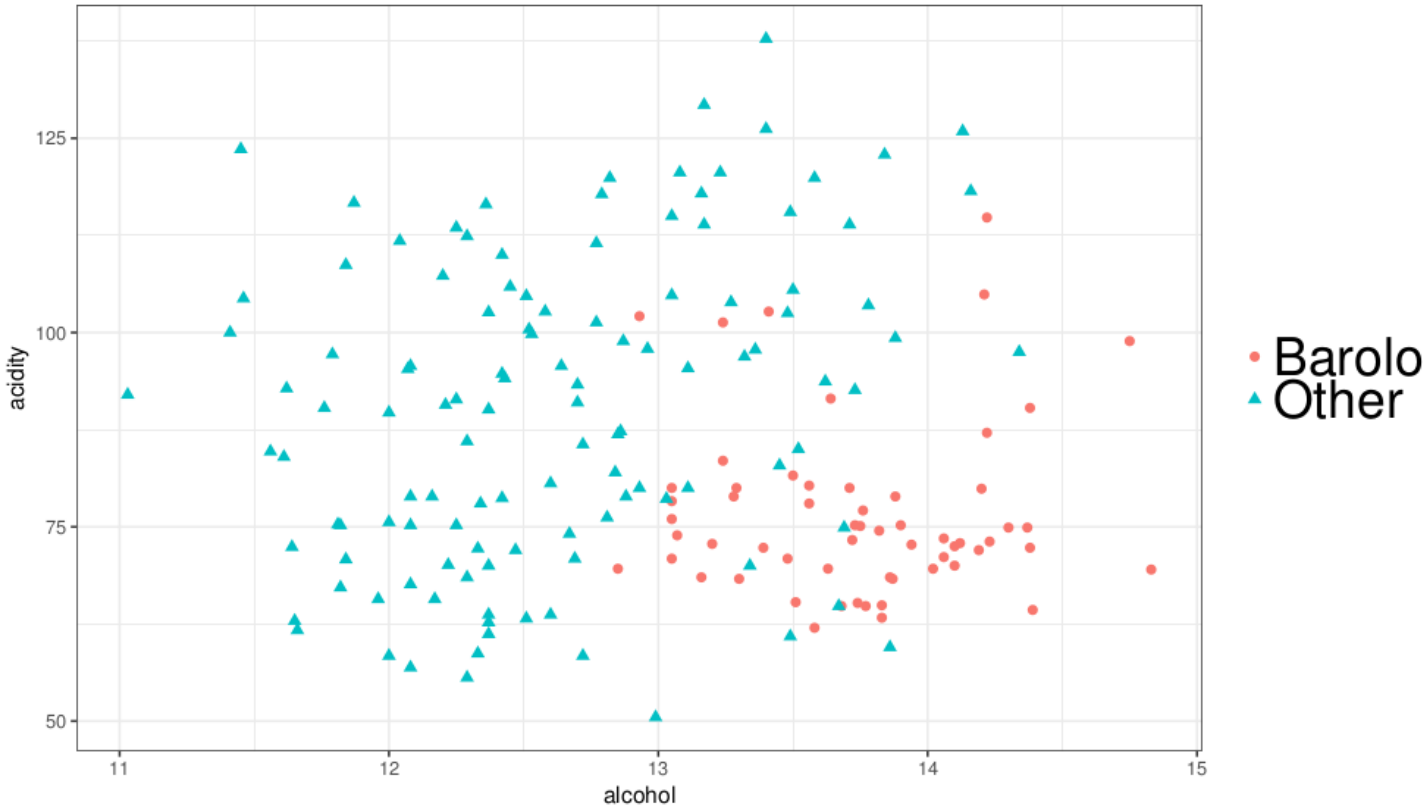
➢ This involve two terms:

- ➢ The class conditional densities $p(x \mid C_k)$, which are the K different distributions $p(x \mid C_1), \ \ldots, \ (x \mid C_K)$ of the K classes.
- ➢ The class prior probabilities $p(C_k)$, which are the proportions $p(C_1), \ \ldots, \ (C_K)$ of the classes in the population.

➢ Of course, both $p(x \mid C_k)$ and $p(C_k)$ will need to be learned, but let's assume that we just know them.

A Generative Model

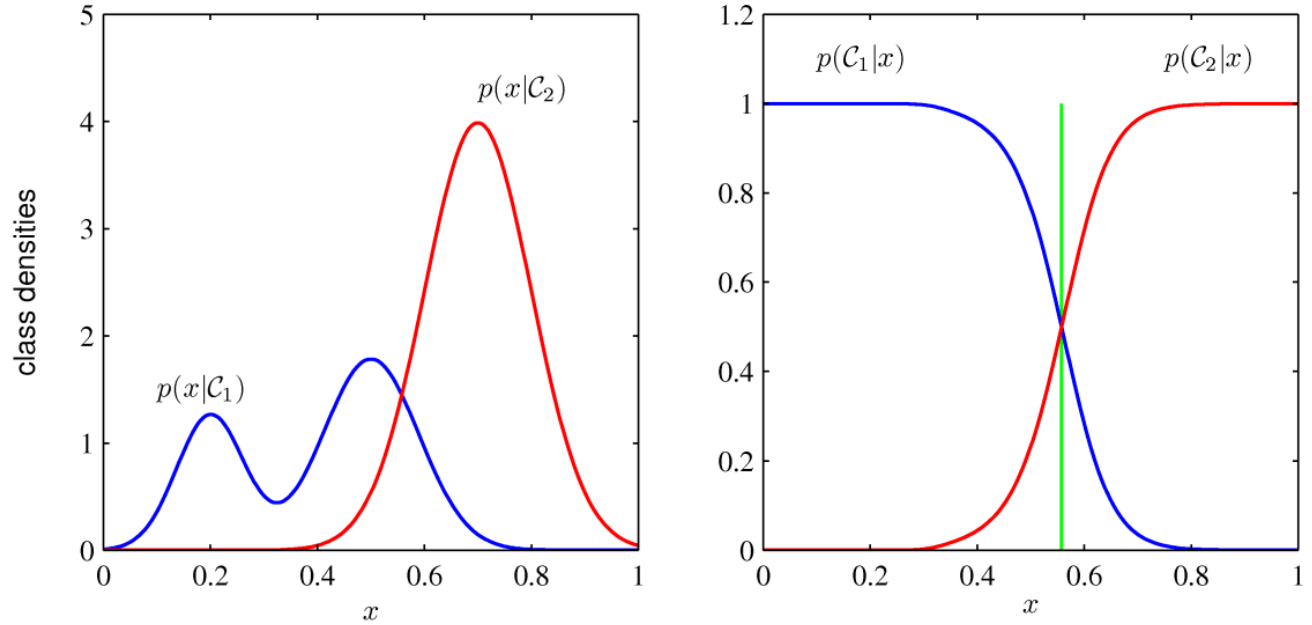
➢ Let's assume we have learned $p(x \mid C_1)$ (density of the Barolos), $p(x \mid C_2)$ (density of the other wines), $p(C_1)$ (proportion of Barolos), $p(C_2)$ (proportion of the other wines).



Estimate Classes

➢ Our main goal is to estimate the probability that the bottle is truly a Barolo. Using Bayes:

$$p(C_1 \mid \boldsymbol{x}) = \frac{p(\boldsymbol{x} \mid C_1)p(C_1)}{p(\boldsymbol{x})} = \frac{p(\boldsymbol{x} \mid C_1)p(C_1)}{p(\boldsymbol{x} \mid C_1)p(C_1) + p(\boldsymbol{x} \mid C_2)p(C_2)}$$



➢ Image: Simplified view on only one variable

Closer Look to the Formula

$$p(C_1 \mid \boldsymbol{x}) = \frac{p(\boldsymbol{x} \mid C_1)p(C_1)}{p(\boldsymbol{x})} = \frac{p(\boldsymbol{x} \mid C_1)p(C_1)}{p(\boldsymbol{x} \mid C_1)p(C_1) + p(\boldsymbol{x} \mid C_2)p(C_2)}$$

➢ It only involves the class-conditional densities and the class proportions. Rewrite:

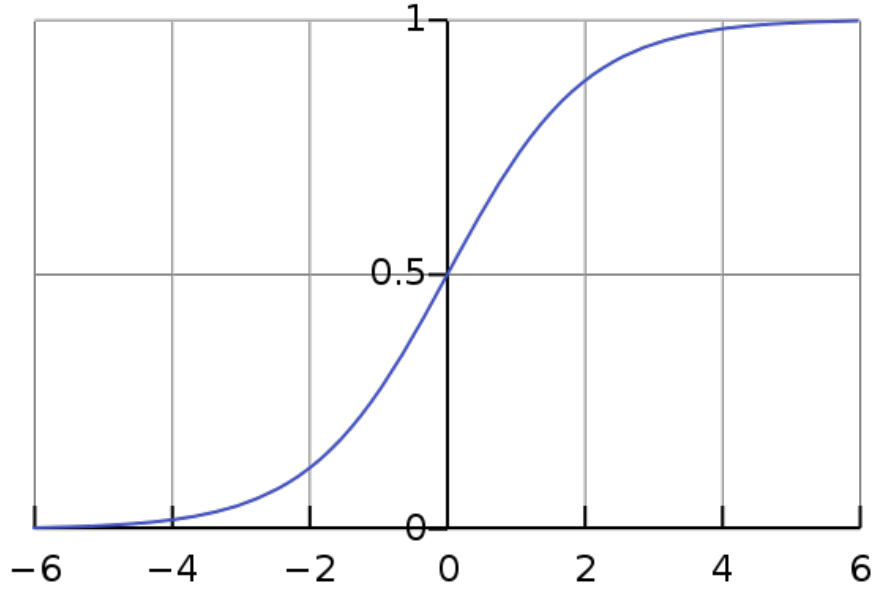
$$p(C_1 \mid \boldsymbol{x}) = \sigma(a(\boldsymbol{x})), \text{ with } a(\boldsymbol{x}) = \ln \frac{p(\boldsymbol{x} \mid C_1)p(C_1)}{p(\boldsymbol{x} \mid C_2)p(C_2)} = \ln \frac{p(\boldsymbol{x}, C_1)}{p(\boldsymbol{x}, C_2)}$$

Closer Look to the Formula

$$p(C_1 \mid \boldsymbol{x}) = \sigma\big(a(\boldsymbol{x})\big), \text{ with } a(\boldsymbol{x}) = \ln \frac{p(\boldsymbol{x} \mid C_1)p(C_1)}{p(\boldsymbol{x} \mid C_2)p(C_2)} = \ln \frac{p(\boldsymbol{x}, C_1)}{p(\boldsymbol{x}, C_2)}$$

➢ With σ being the logistic sigmoid function

$$\sigma(a) = \frac{1}{1 + \exp(-a)} = \frac{e^x}{e^x + 1}$$



The Sigmoid Function

- Why on earth do we need this weird sigmoid function?

- Interpretation: we can interpret $a(x)$ as the amount of evidence about x being a true Barolo, because it's the logarithm of the ratio of probabilities of being true and being fake:
  - Using Bayes's rule, we can see that $a(\boldsymbol{x}) = \ln \frac{p(C_1 \mid \boldsymbol{x})}{p(C_2 \mid \boldsymbol{x})}$

- Unification of the generative and discriminative schools: using the logistic function makes a bridge between the generative methods, and the discriminative ones and neural networks … you will see soon

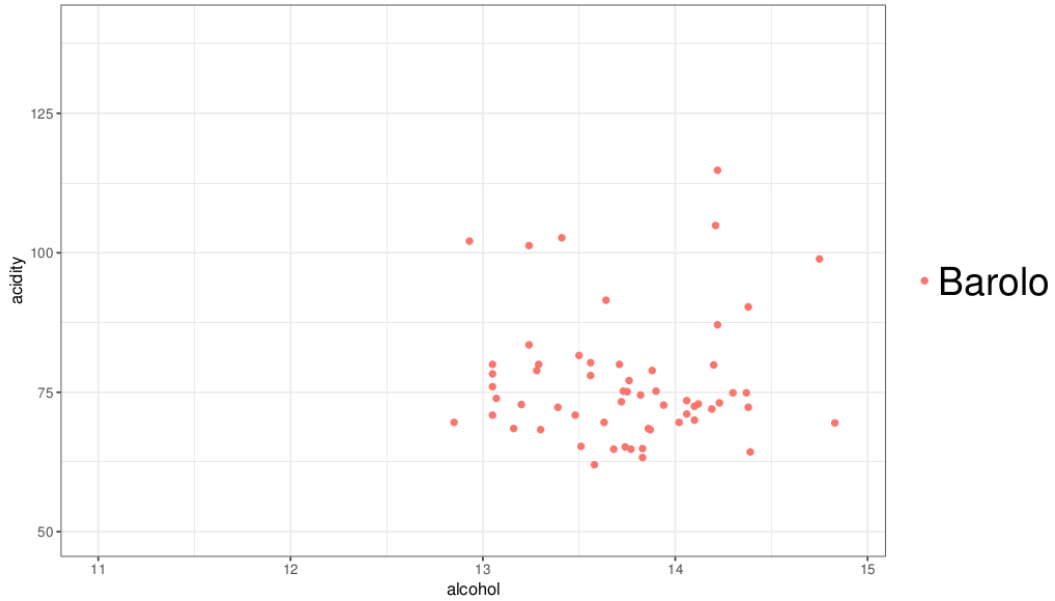
What we have done, what we still have to do...

- **What we have done**.
  - If we know the class-conditional densities $p(x \mid C_k)$ and the class proportions $p(C_k)$, we can compute the posterior probabilities $p(C_k \mid x)$, and use them to classify data.

- **What we still have to do**. How can we find the class-conditional densities and the proportions?
  - We can choose a parametrized model family
  - For that we could use for instance Gaussian Mixture Models
  - Then use MLE to find the best parameters for a Gaussian

Maximum Likelihood Estimators

➢ Let's look at the Barolos:



➢ We now need to fit a Gaussian that describes the data best:

$$N(\boldsymbol{x};\boldsymbol{\mu},\boldsymbol{\Sigma}) = \sqrt{\frac{1}{(2\pi)^2\,|\boldsymbol{\Sigma}|}}\exp\left(-\frac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu})^T\boldsymbol{\Sigma}^{-1}(\boldsymbol{x}-\boldsymbol{\mu})\right)$$

Maximum Likelihood Estimation

- The Maximum Likelihood Estimation (MLE) is a method of estimating the parameters of a model. This estimation method is one of the most widely used.

- The method of maximum likelihood selects the set of values of the model parameters that maximizes the likelihood function. Intuitively, this maximizes the "agreement" of the selected model with the observed data.

- The Maximum-likelihood Estimation gives an unified approach to estimation.

What is a Maximum Likelihood Estimator?

➢ The likelihood that one datapoint was generated by any distribution function is given by

$$l\left(\boldsymbol{x}, \boldsymbol{\theta}\right) = p\left(\boldsymbol{x}; \boldsymbol{\theta}\right)$$

➢ For the entire dataset, we have the likelihood

$$L\left(X, \boldsymbol{\theta}\right) = \prod_{i=1}^{n} p(\boldsymbol{x}^{(i)}; \boldsymbol{\theta})$$

What is a Maximum Likelihood Estimator?

➢ Goal of the MLE is to find θ in such a way that $L\left(X, \theta\right)$ is maximized

$$\theta_{\text{MLE}} = \text{argmax}_{\theta}\, L\left(X, \theta\right)$$

➢ Very often (for simplicity), the log likelihood is used

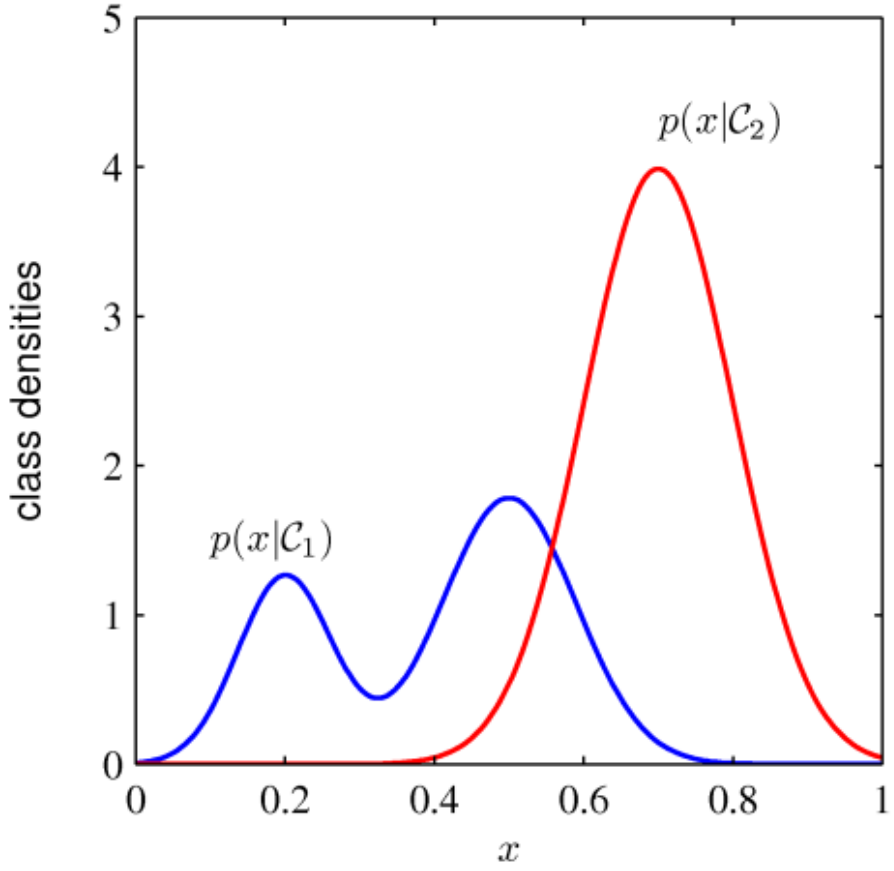
$$\theta_{\text{MLE}} = \text{argmax}_{\theta} \log \prod_{i=1}^{n} p(\boldsymbol{x}^{(i)}; \boldsymbol{\theta}) = \ = \text{argmax}_{\theta} \sum_{i=1}^{n} \log p(\boldsymbol{x}^{(i)}; \boldsymbol{\theta})$$

- ➢ This doesn't change the maximum of the function
- ➢ Eases many calculations

Example: MLE for a Gaussian

- For the sake of simplicity, we only approximate a one-dimensional gaussian for the Barolo, i.e., we just concentrate on one feature

$$\frac{1}{\sigma\sqrt{2\pi}}\exp\left[-\frac{(x-\mu)^2}{2\sigma^2}\right]$$



Example: MLE for a Gaussian

➢ Target: Estimate μ and σ for a Gaussian. The likelihood is

$$L(X\,|\,\theta) = \prod_{i=1}^{n} \frac{1}{\sigma\sqrt{2\pi}} \exp\left[-\frac{\left(x-\mu\right)^2}{2\sigma^2}\right]$$

➢ Taking the log results in an easier version:

$$LL(X\,|\,\theta) = -\frac{1}{2} n \log 2\pi - n\log\sigma - \sum_{i=1}^{n} \frac{\left(x-\mu\right)^2}{2\sigma^2}$$

Example: MLE for a Gaussian

➢ Derivate for μ:

$$\frac{\partial LL}{\partial \mu} = \frac{1}{\sigma^2} \sum_{i=1}^{n} \left(x_i - \mu \right)$$

➢ And for σ^2:

$$\frac{\partial LL}{\partial \sigma^2} = \frac{1}{2\sigma^2} \left[\frac{1}{\sigma^2} \sum_{i=1}^{n} \left(x_i - \mu \right)^2 - n \right]$$

➢ These derivates are 0 if

$$\mu = \frac{1}{n} \sum_{i=1}^{n} x_i \qquad \text{and} \qquad \sigma = \frac{1}{n} \sum_{i=1}^{n} \left(x_i - \mu \right)^2$$

Result

- The Estimator works the same in the multi-dimensional case, except the math gets a bit more iffy (Matrices)

- Now, we have a model for the Barolos
  - We Could now fit a Gaussian over the other wines
  - We calculate our priors, i.e., class prior probabilities, which is simply the ratio of Barolos of all wines in our dataset

- We have everything to compute:

$$p(C_1 \mid \boldsymbol{x}) = \frac{p(\boldsymbol{x} \mid C_1)p(C_1)}{p(\boldsymbol{x})} = \frac{p(\boldsymbol{x} \mid C_1)p(C_1)}{p(\boldsymbol{x} \mid C_1)p(C_1) + p(\boldsymbol{x} \mid C_2)p(C_2)}$$

Keywords

- Generative Models
- Discriminative Models & Logistic Regression
- Information Theory
- Well Behaved Gradients

From generative to discriminative models

➢ We defined a distribution $p(\boldsymbol{x}, C_k)$ over features and classes. We saw how to learn it using the decomposition

$$p(\boldsymbol{x},\ C_k) = p(\boldsymbol{x} \mid C_k)p(C_k)$$

➢ Using this distribution, we were able to compute the posterior probabilities of the classes $p(C_k \mid \boldsymbol{x})$, and to use them for classification.

➢ But, the only thing that we actually need for classification are those posterior probabilities.

➢ **So, could we avoid having to compute $p(x, C_k)$ and directly target the posterior probabilities?**

From generative to discriminative models

➢ The answer to this question is the general rationale of discriminative models:

Learn a way to transform any input vector x into its posterior class distributions

$$p(C_1|x), \ldots, p(C_K|x)$$

From generative to discriminative models

➢ So, we're looking for a way to transform any input vector x into its posterior class distributions $p(C_1 \mid x), \ldots, p(C_K \mid x)$.

➢ For simplicity, let's start with the binary case ($K = 2$).

➢ We have already seen:

$$p(C_1 \mid \boldsymbol{x}) = \sigma(a(\boldsymbol{x})), \text{ with } a(\boldsymbol{x}) = \ln \frac{p(x, C_1)}{p(x, C_2)}$$

From generative to discriminative models

➢ So, with generative models, we have the formula $p(C_1 \mid x) = \sigma\big(a(x)\big)$, where the function $a(x)$ comes from the class proportions and the model $p\big(x, C_k\big)$.

➢ The main idea of discriminative modelling will be to forget that $a(x)$ comes from a generative model, and to rather treat $a(x)$ as an unknown function to be learned from the data.

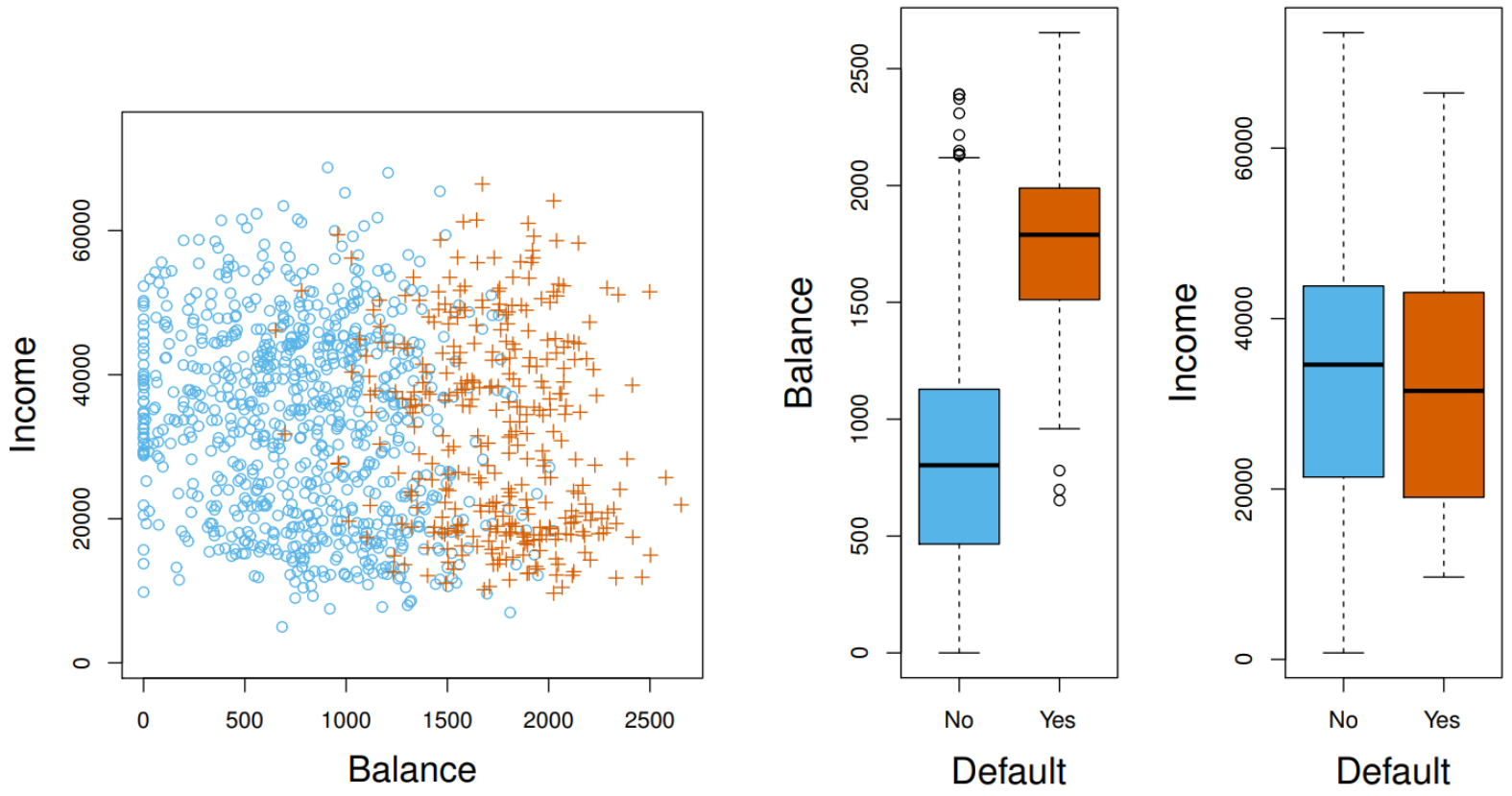
Say Hi to the Big Family of Discriminative Models

➢ The General model is

$$p(C_1 \mid \boldsymbol{x}) = \sigma(a(\boldsymbol{x}))$$

➢ it would be impractically hard to learn any kind of function without making additional assumptions:
- ➢ **Logistic regression** assumes that the function a is linear
- ➢ **Advanced Machine Learning** assumes that the function a is a composition of many linear and nonlinear functions
- ➢ **Gaussian processes** and **nonparametric methods** more generally, try to make as few assumptions as possible regarding a

Example for Logistic Regression: Credit Card Default



Logistic Regression

➢ In Logistic regression we model $a(\boldsymbol{x})$ as a linear regression

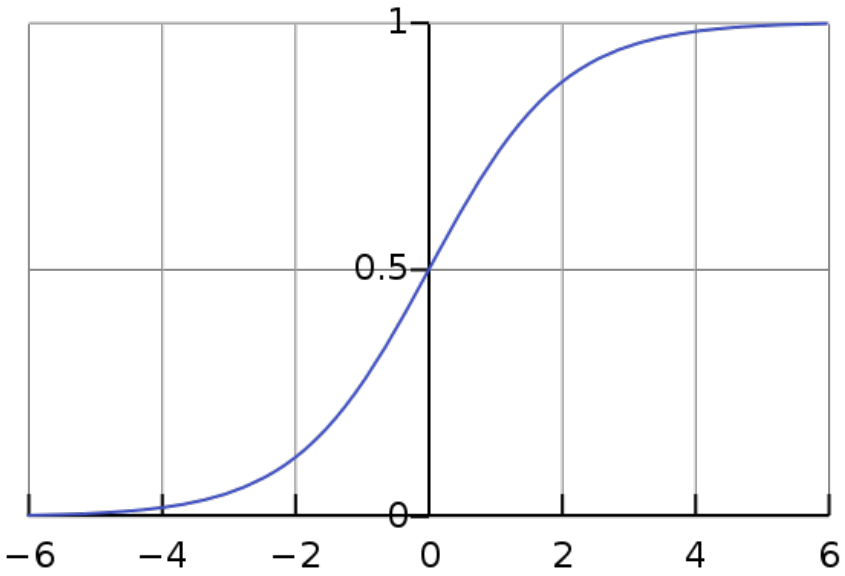
$\beta_0 + \beta_1 x$

➢ Put everything together, with $\boldsymbol{\theta} = \left(\beta_0, \beta_1\right)^T$:

$$h_{\boldsymbol{\theta}}(\boldsymbol{x}) = p\left(y \middle| \boldsymbol{x}, \boldsymbol{\theta}\right) = \frac{e^{\beta_0 + \beta_1 X}}{1 + e^{\beta_0 + \beta_1 X}}$$

Learning Goal

$$\sigma(x) = \frac{e^x}{e^x + 1}$$

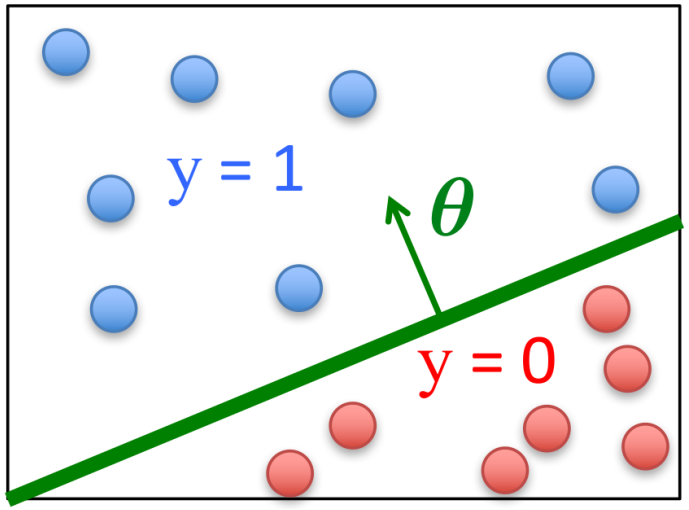


➢ Our Model:

$h_\theta(\boldsymbol{x}) = p\left(y \middle| \boldsymbol{x}, \boldsymbol{\theta}\right) = \sigma(\boldsymbol{\theta}^T \boldsymbol{x})$

➢ $\boldsymbol{\theta}^T \boldsymbol{x}$ should be large negative for negative instances

➢ $\boldsymbol{\theta}^T \boldsymbol{x}$ should be large positive for positive instances



➢ Select a threshold t and
  ➢ Predict $y = 1$ if $p\left(y \middle| \boldsymbol{x}, \boldsymbol{\theta}\right) \geq t$
  ➢ Predict $y = 0$ if $p\left(y \middle| \boldsymbol{x}, \boldsymbol{\theta}\right) < t$

Logistic Regression Objective Function

➢ We cannot just use the least squared approach as with linear regression

$$J(\theta) = \frac{1}{n} \sum_{i}^{n} \left(h_\theta\left(x^{(i)}\right) - y^{(i)} \right)^2$$

➢ Since the logistic regression model with the sigmoid function results in a non-convex optimization problem.
➢ And due to other problems would not lead to the optimal parameter set

Remember MLE

- The Maximum Likelihood Estimation (MLE) is a method of estimating the parameters of a model. This estimation method is one of the most widely used.

- The method of maximum likelihood selects the set of values of the model parameters that maximizes the likelihood function. Intuitively, this maximizes the "agreement" of the selected model with the observed data.

- And it worked like a charm with Gaussian ☺

Computing the Likelihood

➢ Let's calculate the log-likelihood:

$$\ell(a) = \sum_{i=1}^{n} \log p\left(y^{(i)} \middle| \boldsymbol{x}^{(i)}; \boldsymbol{\theta}\right) = \; = \sum_{i=1}^{n} \log \left[h_\theta\left(\boldsymbol{x}^{(i)}\right)^{y^{(i)}} \cdot \left(1 - h_\theta\left(\boldsymbol{x}^{(i)}\right)\right)^{1-y^{(i)}} \right] =$$

$$= \sum_{i=1}^{n} \left[y^{(i)} \log h_\theta\left(\boldsymbol{x}^{(i)}\right) + \left(1 - y^{(i)}\right) \log\left(1 - h_\theta\left(\boldsymbol{x}^{(i)}\right)\right) \right]$$

Computing the Likelihood

➢ We have

$$\sum_{i=1}^{n}\left[y^{(i)}\log h_\theta\left(\boldsymbol{x}^{(i)}\right)+\left(1-y^{(i)}\right)\log\left(1-h_\theta\left(\boldsymbol{x}^{(i)}\right)\right)\right]$$

➢ We want to maximize this function, which is equivalent to minimizing its opposite, which is called the cross-entropy loss:

$$J(\theta)=-\sum_{i=1}^{n}\left[y^{(i)}\log h_\theta\left(\boldsymbol{x}^{(i)}\right)+\left(1-y^{(i)}\right)\log\left(1-h_\theta\left(\boldsymbol{x}^{(i)}\right)\right)\right]$$

➢ The cross-entropy loss is the most commonly used loss for neural networks, and is a way of doing maximum likelihood without necessarily saying it.

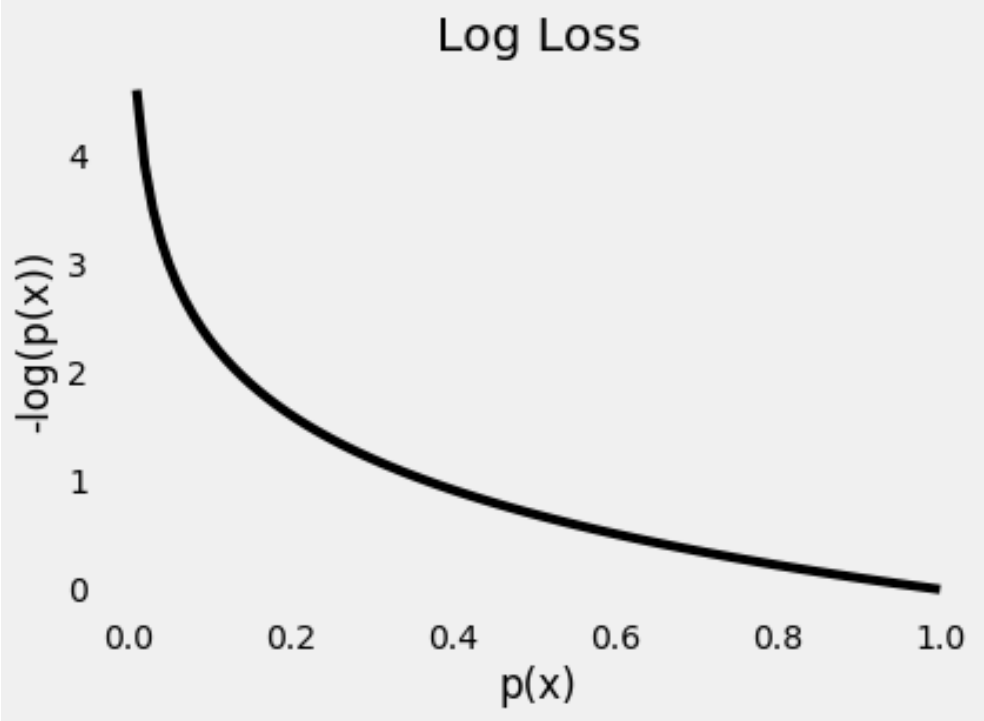
Intuition Behind the Objective

$$J(\theta) = -\sum_{i=1}^{n}\left[y^{(i)}\log h_\theta\left(\boldsymbol{x}^{(i)}\right) + \left(1-y^{(i)}\right)\log\left(1-h_\theta\left(\boldsymbol{x}^{(i)}\right)\right)\right]$$

- Depending on y, the loss function assumes these values:

$$J(\theta) = \begin{cases} -\log\left(h_\theta(\boldsymbol{x})\right) & \text{for} \quad y=1 \\ -\log\left(1-h_\theta(\boldsymbol{x})\right) & \text{for} \quad y=0 \end{cases}$$

- Effect: Extremely negative with wrong classifications
- Trained with gradient descent methods



Keywords

- Generative Models
- Discriminative Models & Logistic Regression
- Information Theory
- Well Behaved Gradients

Cross Entropy Loss

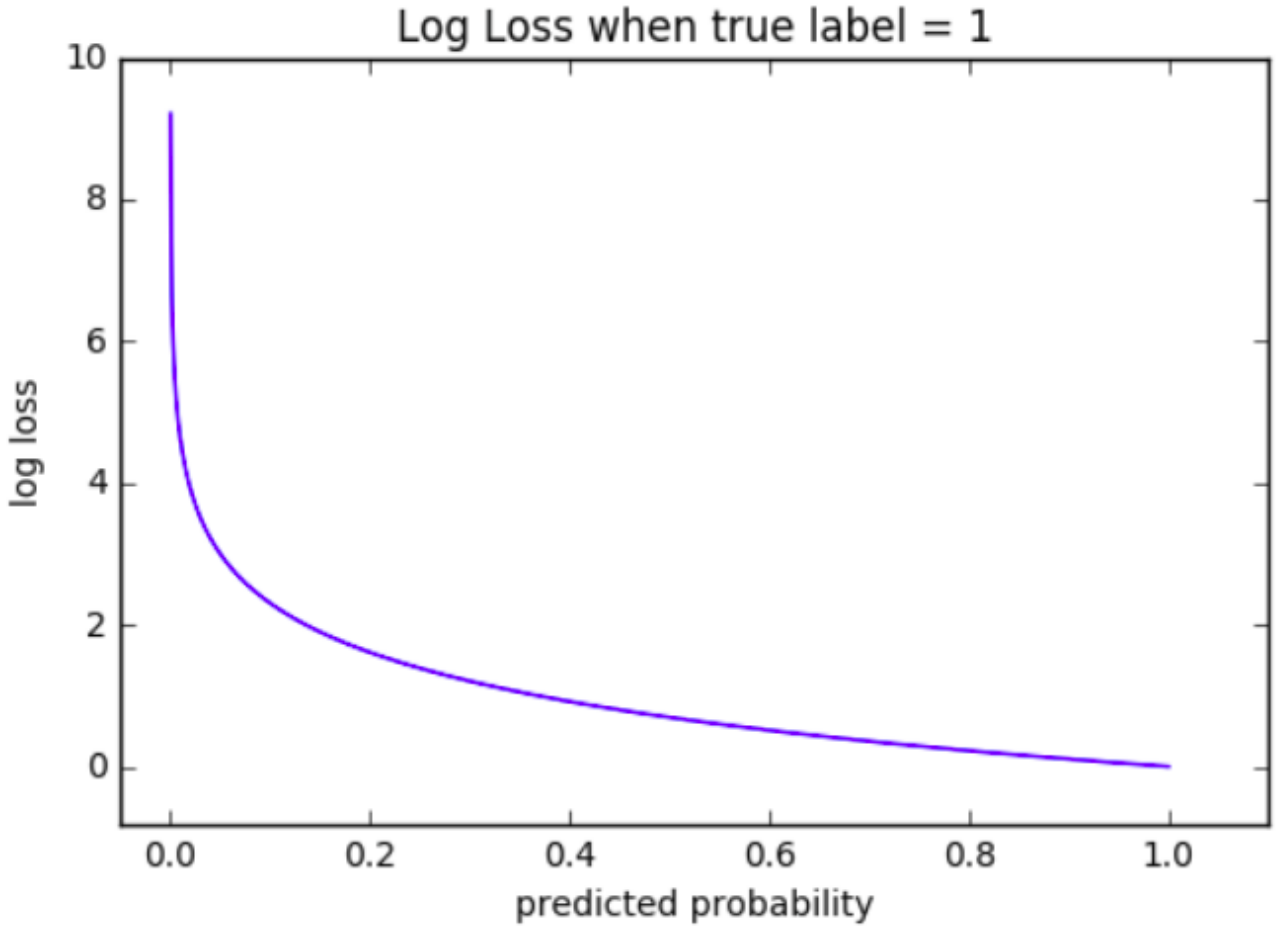
➢ Minimizing the cross-entropy loss corresponds to maximize the log likelihood (but comes from a different theoretical background):

$$J(\theta) = -\mathbb{E}\left[p(y \mid \boldsymbol{x}; \boldsymbol{\theta})\right]$$

➢ In case of our 2-case logistic regression:

$$-\frac{1}{n}\sum_{i=1}^{n}\left[y^{(i)}\log h_{\boldsymbol{\theta}}(\boldsymbol{x}) + \left(1 - y^{(i)}\right)\log\left(1 - h_{\boldsymbol{\theta}}(\boldsymbol{x})\right)\right]$$

Cross Entropy Loss



Information Theory

- To fully understand the origins of the most common loss functions, we have to resort back to the basics of statistical learning

- The key assumption is that our dataset $X = \{x^{(1)}, \ldots, x^{(m)}\}$ is drawn independently from the true but unknown data generating distribution p_{data}

- Now, let $p_{model}(\boldsymbol{x}; \boldsymbol{\theta})$ be a parametric family of probability distributions

Information Theory

- Intuition
  - The occurrence of an extremely unlikely event is more informative than the occurrence of a very likely event
    - There is no pandemic outbreak – is not very informative
    - There is a pandemic outbreak and you should stay home => very informative
- To quantify this intuition and need a function fulfilling:
  - Likely events should have low information content
    - Extreme case: Guaranteed events shall have no information content
  - Less likely events should have higher information content.
  - Independent events should have additive information.
    - Finding out that a tossed coin has come up as heads twice should convey twice as much information as finding out that a tossed coin has come up as heads once.

Self-Information

➢ In order to satisfy all three of these properties, we define the self-information of an event $X = x$ to be

$$-\log P(X = x)$$

➢ The information content is measured in
- ➢ **nats** … when using the base e
- ➢ **bits** or **shannons** … when using the base 2

➢ To measure the uncertainty for an entire distribution **(Shannon Entropy)**

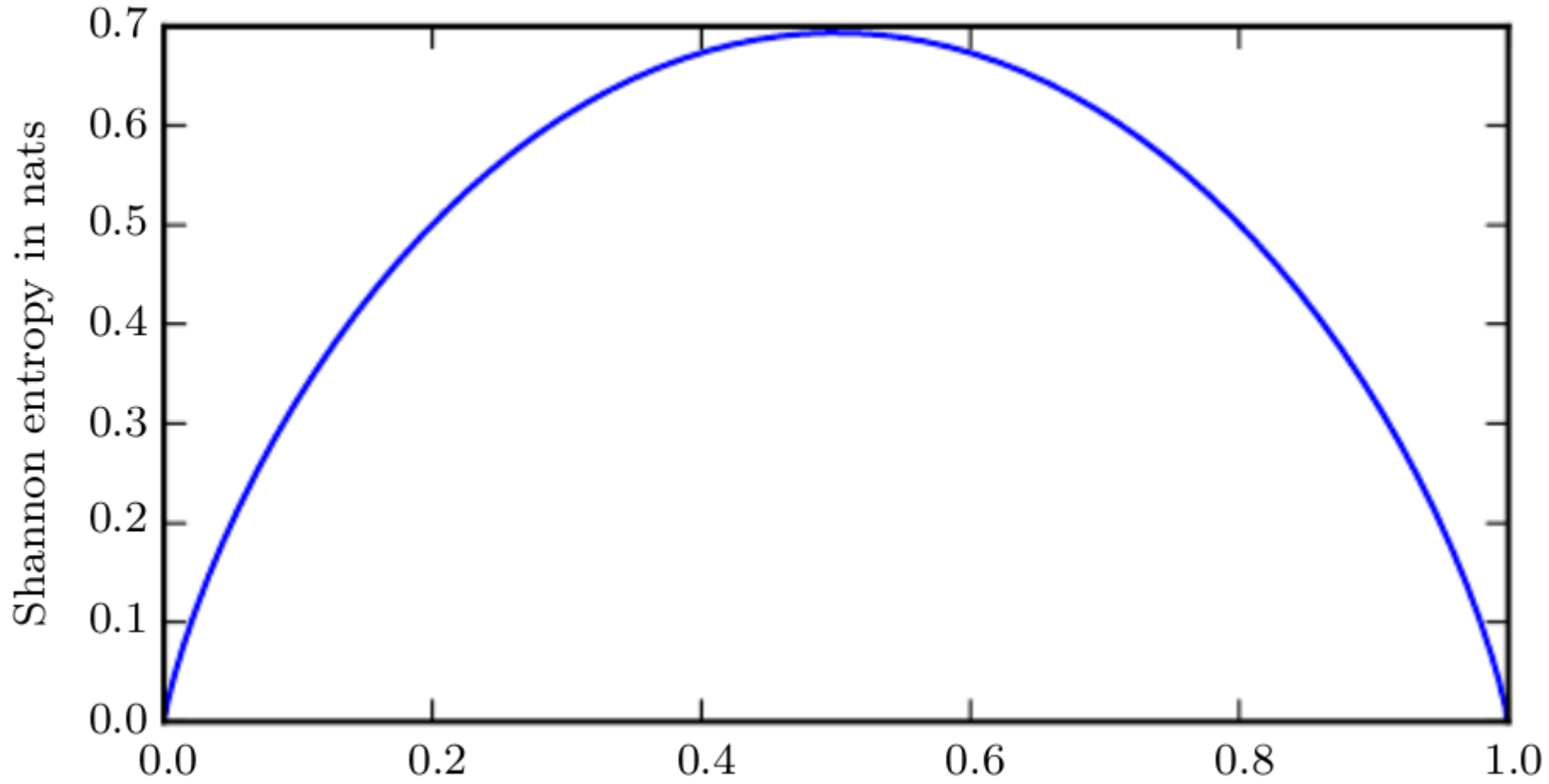
$$H(x) = \mathbb{E}_{x\sim P}\big[I(x)\big] = -\mathbb{E}_{x\sim P}[\log P(X = x)]$$

Shannon Entropy

$$H(x) = \mathbb{E}_{x\sim P}\big[I(x)\big] = -\mathbb{E}_{x\sim P}[\log P(X = x)]$$

- Expected amount of information of an event drawn from P
- It gives a lower bound on the number of bits (if the logarithm is base 2, otherwise the units are different) needed on average to encode symbols drawn from a distribution P.
- Distributions that are nearly deterministic (where the outcome is nearly certain) have low entropy; distributions that are closer to uniform have high entropy.

Shannon Entropy



The Maximum Likelihood Estimator

➢ The maximum likelihood estimator for θ is then defined as

$$\theta_{ML} = \text{argmax}_{\theta} p_{model}\left(X; \theta\right)$$
$$= \text{argmax}_{\theta} \prod_{i=1}^{m} p_{model}(\boldsymbol{x}^{(i)}; \boldsymbol{\theta})$$

➢ The argmax function is invariant to rescaling, thus

$$\theta_{ML} = \text{argmax}_{\theta} \sum_{i}^{m} \log p_{model}(\boldsymbol{x}^{(i)}; \boldsymbol{\theta})$$
$$= \text{argmax}_{\theta} \mathbb{E}_{\boldsymbol{x} \sim \hat{p}_{data}} \log p_{model}(\boldsymbol{x}; \boldsymbol{\theta})$$

Kullback-Leibler Divergence

- If we have two separate probability distributions $P(x)$ and $Q(x)$ over the same random variable x, we can measure how different these two distributions are using the Kullback-Leibler (KL) divergence:

$$D_{KL}\big(P||Q\big) = \mathbb{E}_{x\sim P}\left[\log\frac{P(x)}{Q(x)}\right] = \mathbb{E}_{x\sim P}\big[\log P(x) - \log Q(x)\big]$$

  - In discrete case it can be read as the extra amount of information needed to send a message drawn from P, when we use a code designed for Q.

- Properties:
  - Non-negative
  - 0 if and only if P and Q are the same function
  - Not symmetric! (=> no true distance measure)

Cross Entropy

- Closely related to the KL divergence is the cross-entropy

$$H(P,Q) = H(P) + D_{KL}(P||Q) = -\mathbb{E}_{x\sim P}\log Q(x)$$

- Interpretation very similar to KL divergence:
  - **The cross entropy is the average number of bits needed to encode data coming from a source with distribution p when we use model q**
- While as the KL divergence:
  - **measures the number of EXTRA bits required to encode data coming from a source with distribution p when we use model q**
- Minimizing the cross-entropy with respect to Q is equivalent to minimizing the KL divergence, because Q does not participate in the omitted term.

All Together

➢ Different point of view:

➢ Minimize the dissimilarity between the empirical distribution $\hat{p}_{data}$ (defined by the data) and the model with Kullback-Leibler (KL) distance:

$$D_{KL}\left(\hat{p}_{data},\ p_{model}\right) = \mathbb{E}_{\boldsymbol{x}\sim\hat{p}_{data}}\left[\log\frac{\hat{p}_{data}}{p_{model}(\boldsymbol{x};\boldsymbol{\theta})}\right] = \mathbb{E}_{\boldsymbol{x}\sim\hat{p}_{data}}\left[\log\hat{p}_{data} - \log p_{model}(\boldsymbol{x};\boldsymbol{\theta})\right]$$

➢ We have just seen, that minimizing this term is equivalent to minimize the cross-entropy:

$$-\mathbb{E}_{\boldsymbol{x}\sim\hat{p}_{data}}\left[\log p_{model}(\boldsymbol{x};\boldsymbol{\theta})\right]$$

➢ Minimizing **KL** is equivalent to minimizing the **cross-entropy** or maximizing the **log-likelihood**

About the Gradient

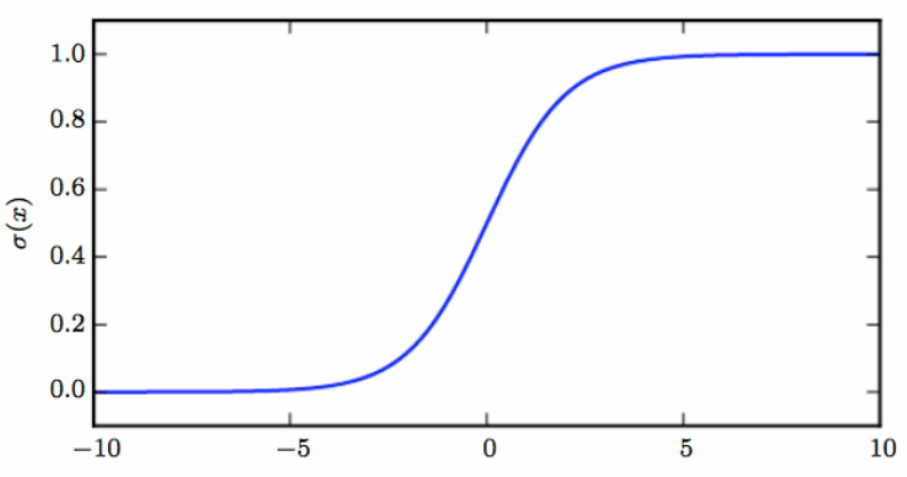
- Gradient must be large and predictable enough to serve as good guide to the learning algorithm
- Functions that **saturate** (become very flat) undermine this
  - Because the gradient becomes very small
  - Happens when activation functions producing output of hidden/output units saturate

- **Negative log-likelihood** helps avoid saturation problem for many models
  - Many output units involve exp functions that saturate when its argument is very negative
  - Log function in Negative log likelihood cost function undoes exp of some units

Keywords

- Generative Models
- Discriminative Models & Logistic Regression
- Information Theory
- Well Behaved Gradients

Revisit: Sigmoid Units

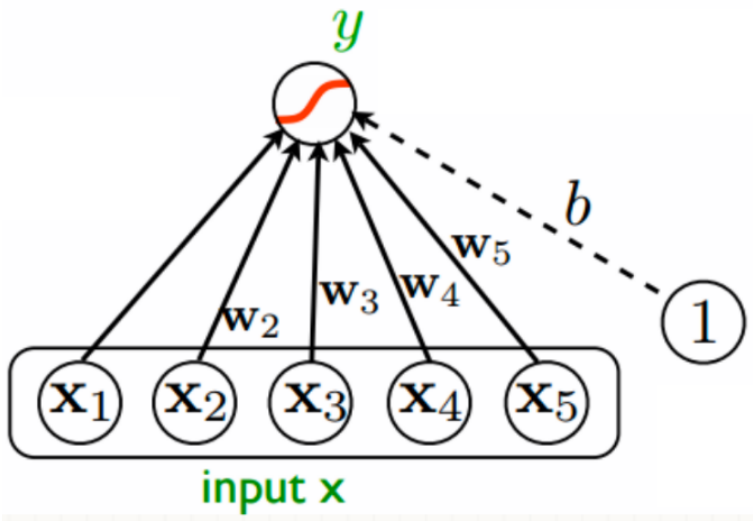


➢ Sigmoid always gives a gradient

$\hat{y} = \sigma(\boldsymbol{w}^T \boldsymbol{h} + b)$

with

$$\sigma(x) = \frac{1}{1 + \exp(-x)}$$



➢ Sigmoid Unit has two components:

- ➢ A linear layer to compute

$z = \boldsymbol{w}^T \boldsymbol{h} + b$

- ➢ A sigmoid activation function to convert z into a probability

Cross-Entropy

➢ Using sigmoid directly:

$$p(y|\theta) = \prod_{n=1}^{N} \sigma(\theta^T x_n)^{y_n} \cdot \left(1 - \sigma(\theta^T x_n)\right)^{1-y_n}$$

➢ With the loss function:

$$J(\theta) = -\ln p(y|\theta) = -\sum_{n=1}^{N} \left(y_n \ln\left(\sigma(\theta^T x_n)\right) + (1 - y_n) \ln\left(1 - \sigma(\theta^T x_n)\right) \right)$$

➢ Problem:
- ➢ Sigmoid saturates to 0 or 1 when z becomes very negative/positive
- ➢ Gradient can shrink to too small to be useful for learning, whether model has correct or incorrect answer

Different Formulation

➢ At the end of the day we need to produce a number which represents $\hat{y} = P(y = 1 \mid \boldsymbol{x})$; thus lies between 0 and 1

➢ Therefore, let's predict a number

$z = \log \tilde{P}(y = 1 \mid \boldsymbol{x})$

with $\tilde{P}$ being an unnormalized random distribution:

$\log \tilde{P}(y) = yz$ $\qquad\qquad$ $\tilde{P}(y) = \exp(yz)$

➢ Normalizing leads to a real distribution:

$$P(y) = \frac{\exp(yz)}{\sum_{y'=\{0,1\}} \exp(y'z)} \qquad\qquad P(y) = \sigma\big((2y-1)z\big)$$

Different Formulation

➢ At the end of the day we need to produce a number which represents $\hat{y} = P(y = 1 \mid \boldsymbol{x})$; thus lies between 0 and 1

➢ Therefore, let's predict a number

$$z = \log \tilde{P}(y = 1 \mid \boldsymbol{x})$$

with $\tilde{P}$ being an unnormalized random distribution:

$$\log \tilde{P}(y) = yz \qquad \tilde{P}(y) = \exp(yz)$$

➢ Normalizing leads to a real distribution:

$$P(y) = \frac{\exp(yz)}{\sum_{y'=\{0,1\}} \exp(y'z)} \qquad P(y) = \sigma\big((2y-1)z\big)$$

$$\log \tilde{P}(y=1) = z$$
$$\log \tilde{P}(y=0) = 0$$

$$\tilde{P}(y=1) = e^z$$
$$\tilde{P}(y=0) = e^0 = 1$$

$$P(y=1) = \frac{e^z}{1+e^z}$$
$$\mathrm{P}(y=0) = \frac{1}{1+e^z}$$

$$\begin{cases} \sigma(z) = \frac{1}{1+e^{-z}} = \frac{e^z}{1+e^z} & \text{for } y = 1 \\ \sigma(-z) = \frac{1}{1+e^z} & \text{for } y = 0 \end{cases}$$

Sigmoid with Softplus

➢ Using

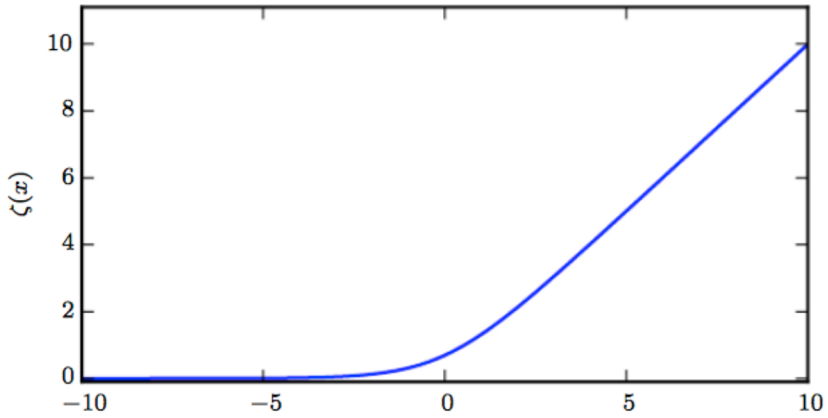
$$P(y \mid z) = \sigma\Big((2y-1)z\Big)$$

➢ The Loss function can be defined as:

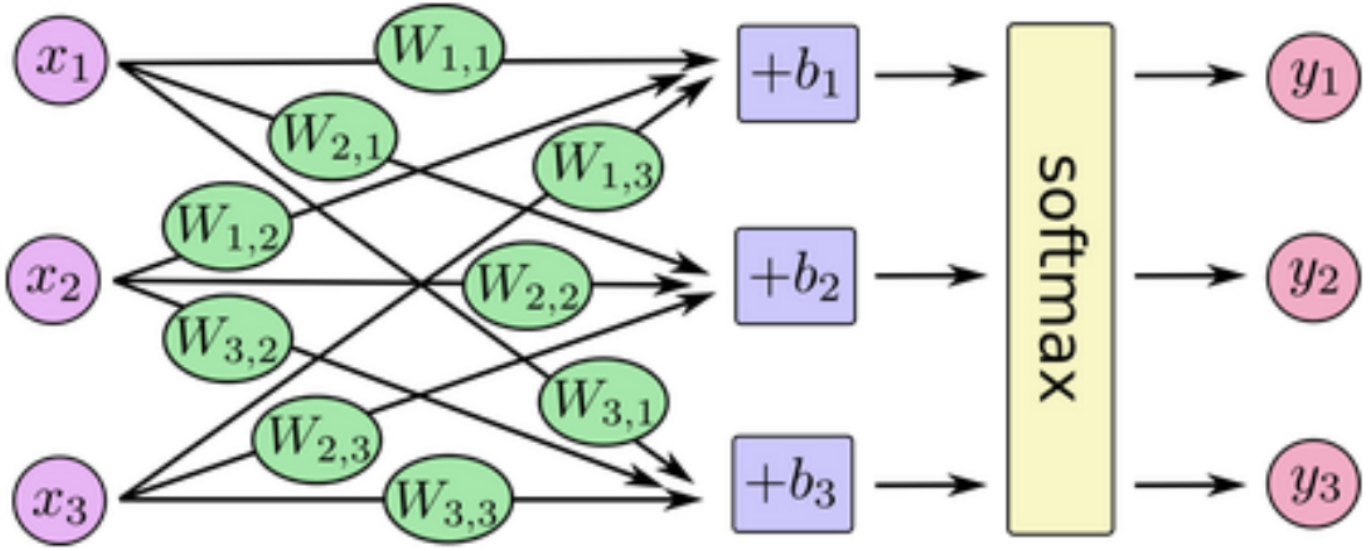
$$J(\theta) = -\log P(y \mid \boldsymbol{x}) = -\log\sigma\Big((2y-1)z\Big) = \zeta\Big((1-2y)z\Big)$$

$$\zeta(x) = \log\big(1+\exp(x)\big) \approx x^{+} = \max(0,x)$$

- ➢ With ζ being the softplus function
- ➢ only when saturates when $(1-2y)z$ is very negative
- ➢ That only happens, when the model already has the right answer



Revisit the Softmax Function



Log-likelihood of the Softmax Function

- As a general rule of thumb, exponentiations should be undone by the cost function
- Thus, the log-likelihood is a natural candidate for the loss function

$$\text{logsoftmax}(\boldsymbol{z})_i = z_i - \log \sum_j \exp\left(z_j\right)$$

- Input z_i always has a direct contribution and can never saturate
  - This first term encourages z_i being pushed up
  - The second term encourages entire $\boldsymbol{z}$ being pushed down

Second Term

➢ We consider the term

$$\log \sum_j \exp\left(z_j\right)$$

➢ Can be approximated to $\max_j z_j$ (when largest z_j is significantly larger than all other z_k)

➢ Costs penalizes most active incorrect prediction

➢ In case of a correct prediction, then z_i and $\log \sum_j \exp\left(z_j\right)$ will roughly cancel out.

More about the Softmax Function

➢ Overall, unregularized maximum likelihood will drive the model to learn parameters that drive the softmax to predict the fraction of counts of each outcome observed in the training set

$$\text{softmax}\left(\boldsymbol{z}\left(\boldsymbol{x};\boldsymbol{\theta}\right)\right)_i \approx \frac{\sum_{j=1}^{m} \mathbf{1}_{y^{(j)}=i,\boldsymbol{x}^{(j)}=\boldsymbol{x}}}{\sum_{j=1}^{m} \mathbf{1}_{\boldsymbol{x}^{(j)}=\boldsymbol{x}}}$$

➢ Objective functions that do not use a log to undo the $\exp$ of softmax fail to learn when argument of $\exp$ becomes very negative, causing gradient to vanish

➢ Especially the squared error is a poor loss function for softmax units

Other Output Types

- Linear, Sigmoid and Softmax output units are the most common
- Neural networks can generalize to any kind of output layer
- Principle of maximum likelihood provides a guide for how to design a good cost function for any output layer
- If we define conditional distribution $p(\boldsymbol{y} \mid \boldsymbol{x}; \boldsymbol{\theta})$, principle of maximum likelihood suggests we use $-\log p(\boldsymbol{y} \mid \boldsymbol{x}; \boldsymbol{\theta})$ for our cost function

Keywords

- Generative Models
- Discriminative Models & Logistic Regression
- Information Theory
- Well Behaved Gradients

How’s my teaching?

https://www.admonymous.co/lukasgalke